Client-side Technologies

Dr. Niveen Nasr El-Den
iTi

Day 6

JavaScript Fundamentals cont.

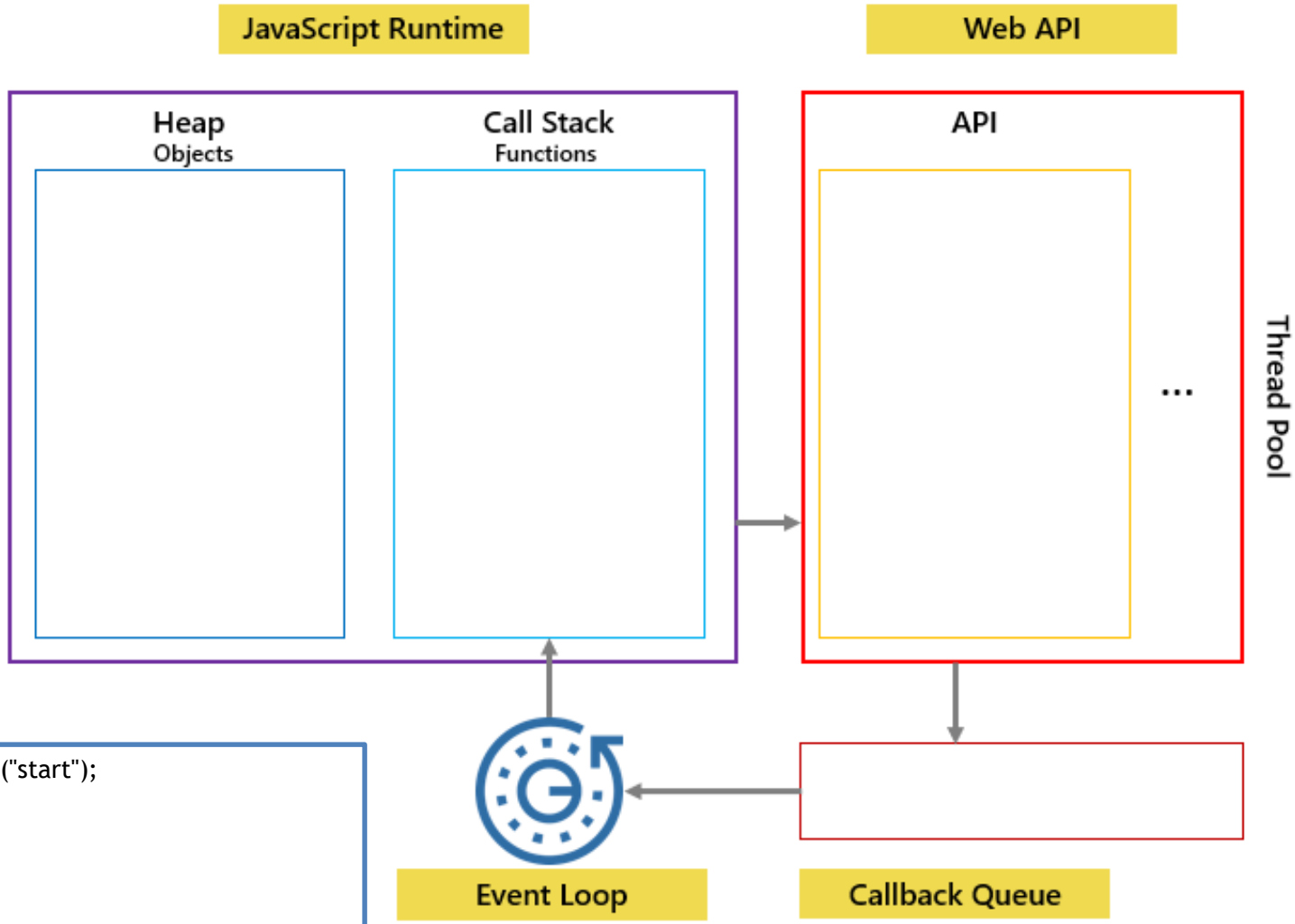
Browser Object Model

BOM

Example

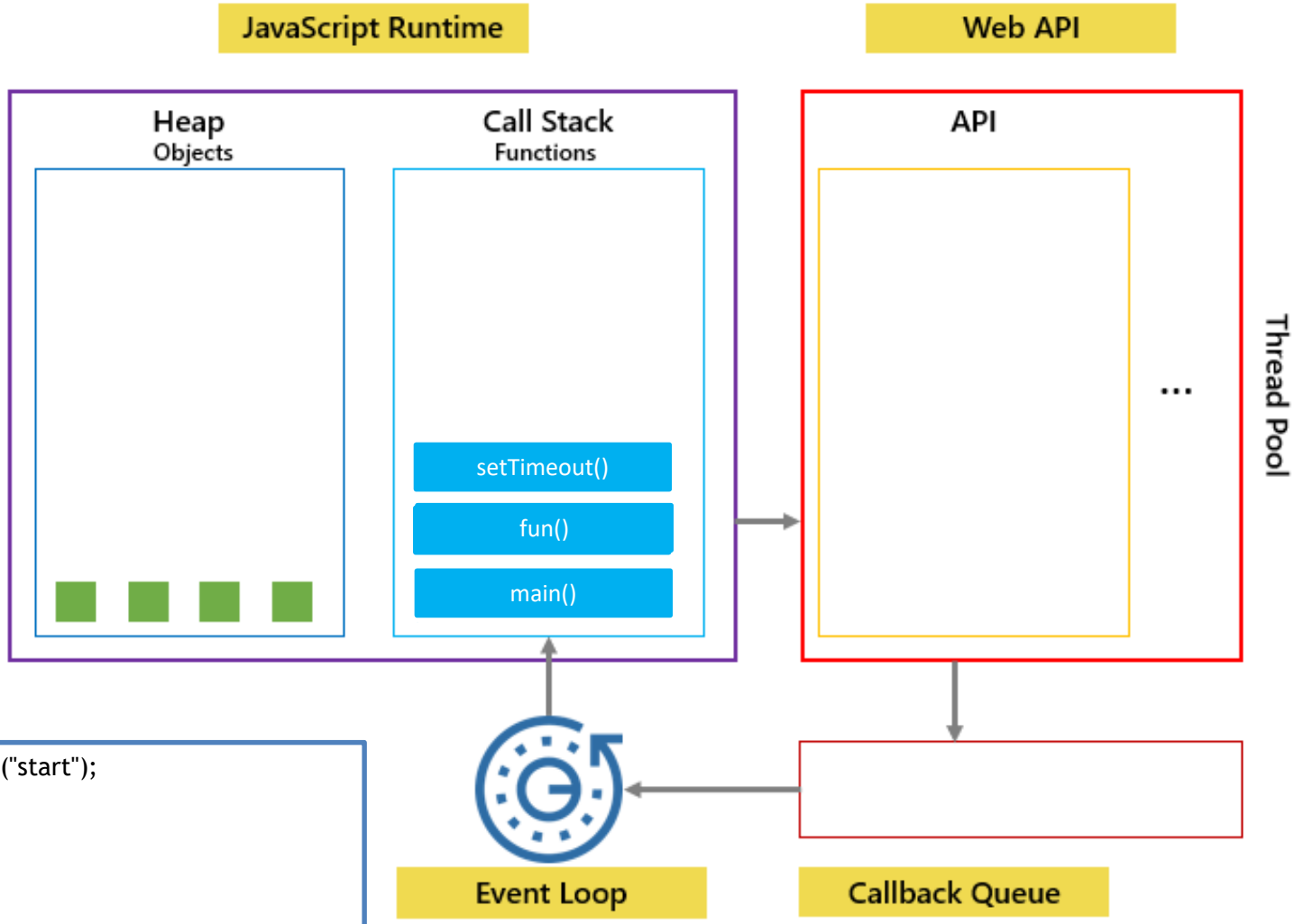
```
console.log("start");

wait5sec();

fun();

console.log("end");

setTimeout(function () {
    console.log("timeout after 1 sec");
}, 1000)
```

```
function fun() {
    setTimeout(function () {
        console.log("timeout immediately");
    }, 0);
}
```



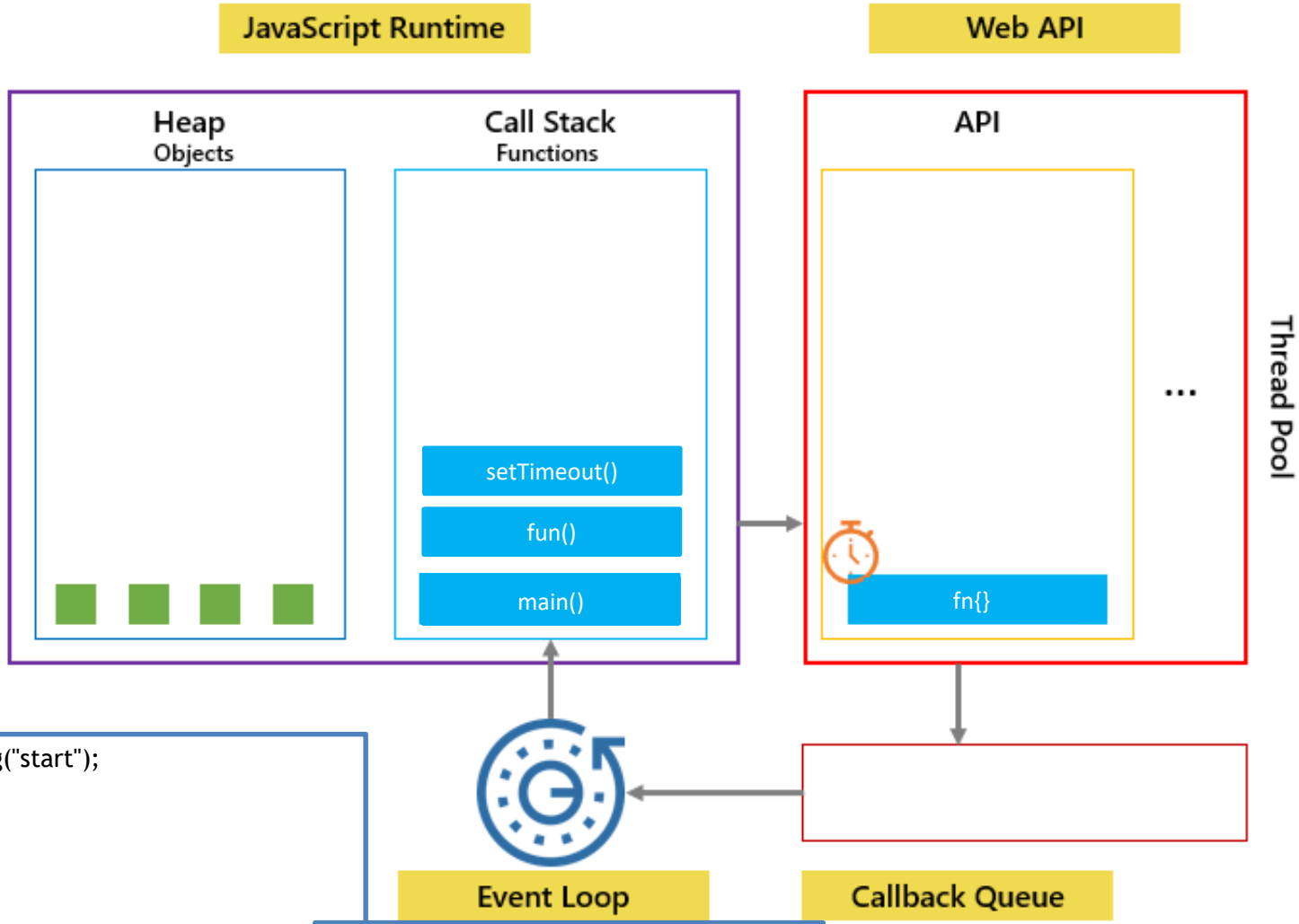

```
console.log("start");

wait5sec();

fun();

console.log("end");

setTimeout(function () {
    console.log("timeout after 1 sec");
}, 1000)
```

```
function fun() {
    setTimeout(function () {
        console.log("timeout immediately");
    }, 0);
}
```



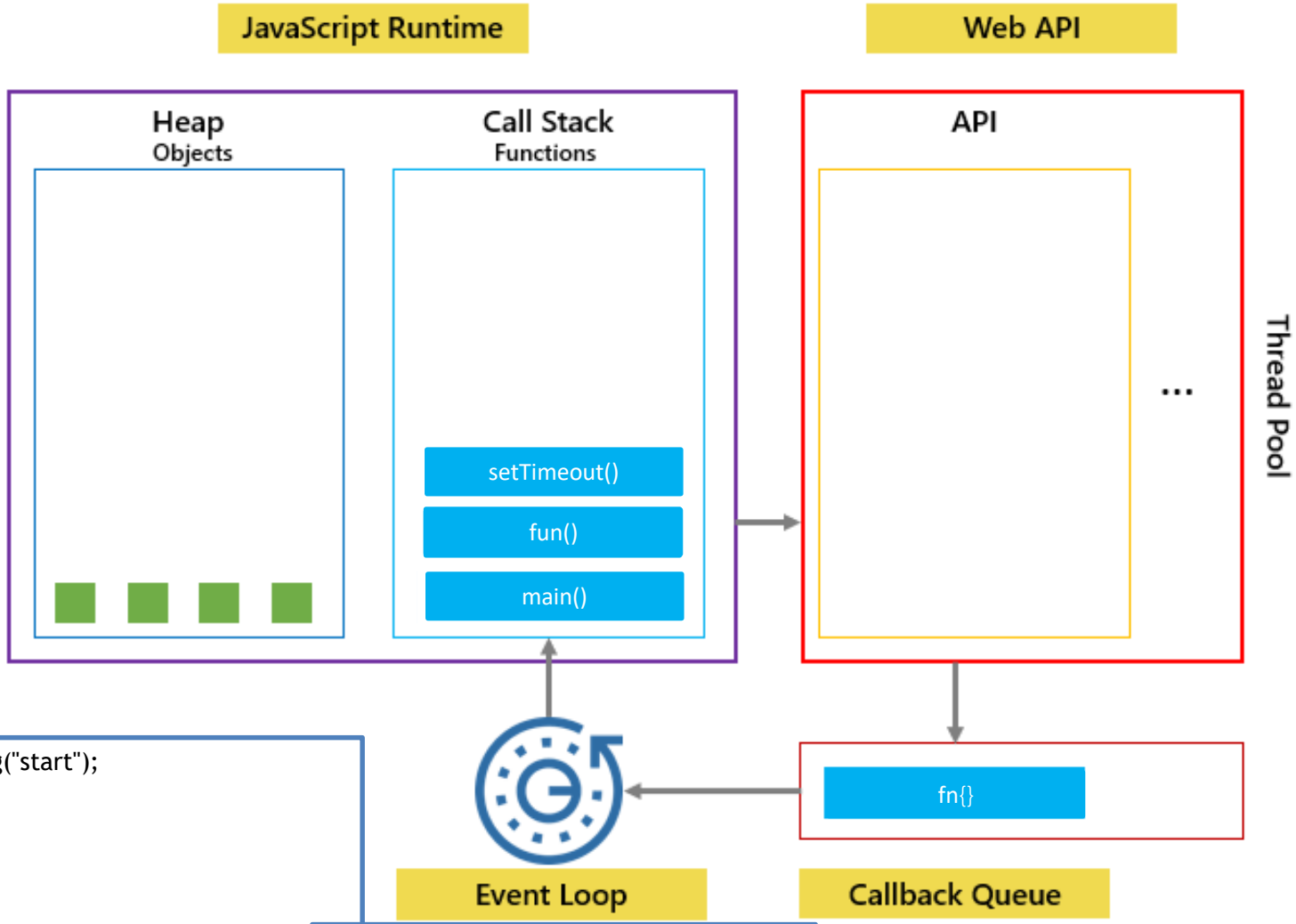

```
console.log("start");

wait5sec();

fun();

console.log("end");

setTimeout(function () {
    console.log("timeout after 1 sec");
}, 1000)
```

```
function fun() {
    setTimeout(function () {
        console.log("timeout immediately");
    }, 0);
}
```



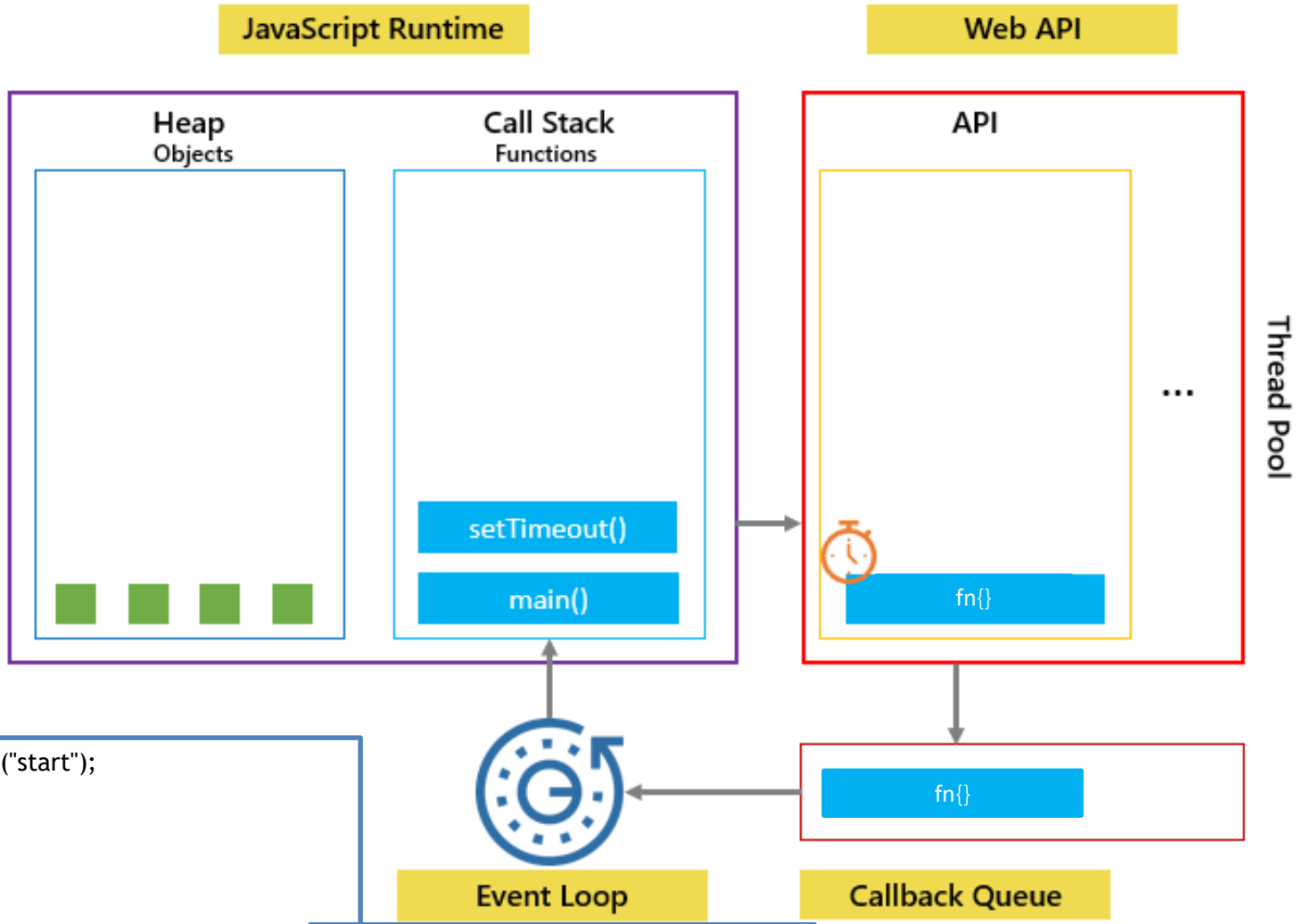

```
console.log("start");

wait5sec();

fun();

console.log("end");

setTimeout(function () {
    console.log("timeout after 1 sec");
}, 1000)
```

```
function fun() {
    setTimeout(function () {
        console.log("timeout immediately");
    }, 0);
}
```



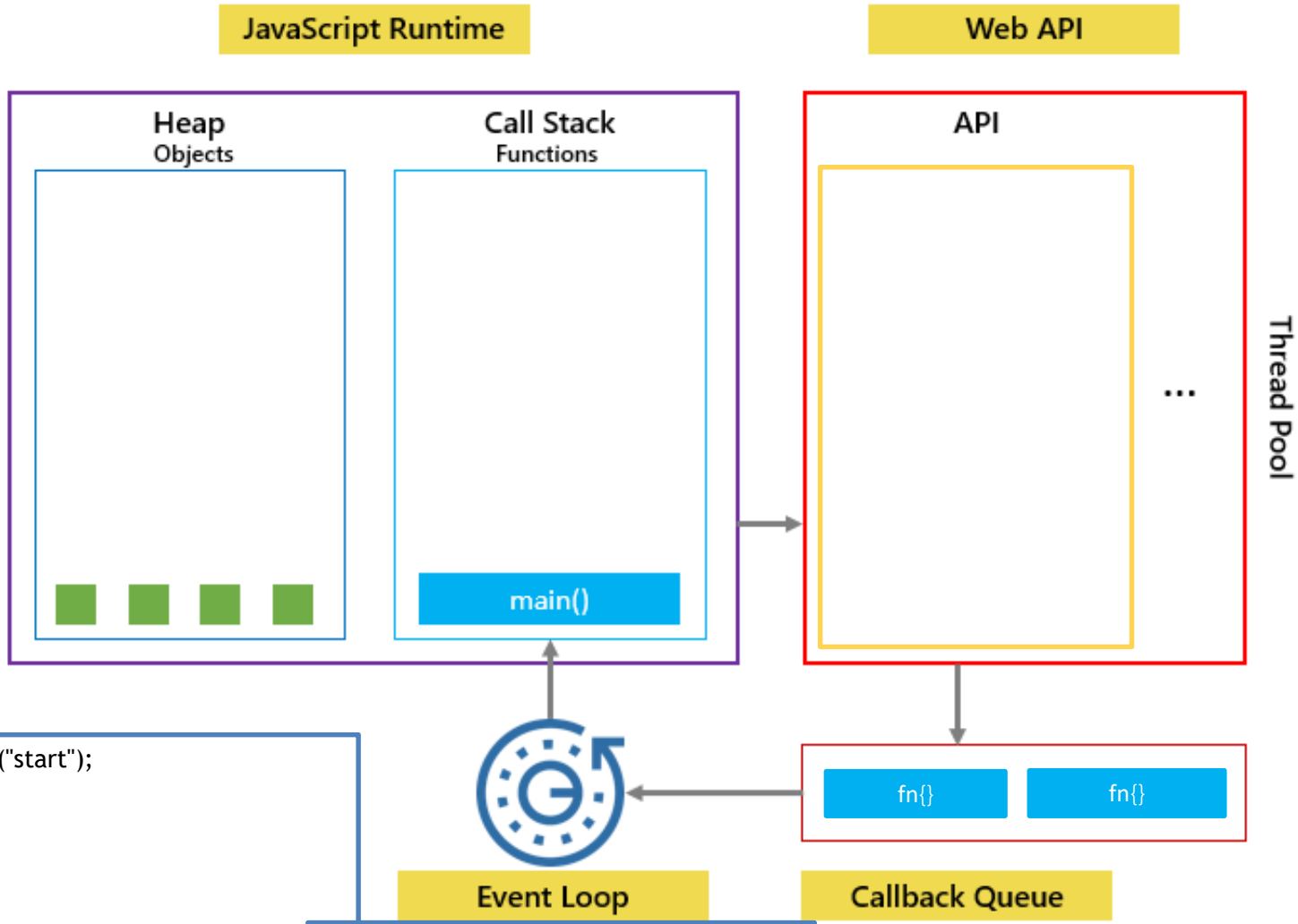

```
console.log("start");

wait5sec();

fun();

console.log("end");

setTimeout(function () {
    console.log("timeout after 1 sec");
}, 1000)
```

```
function fun() {
    setTimeout(function () {
        console.log("timeout immediately");
    }, 0);
}
```



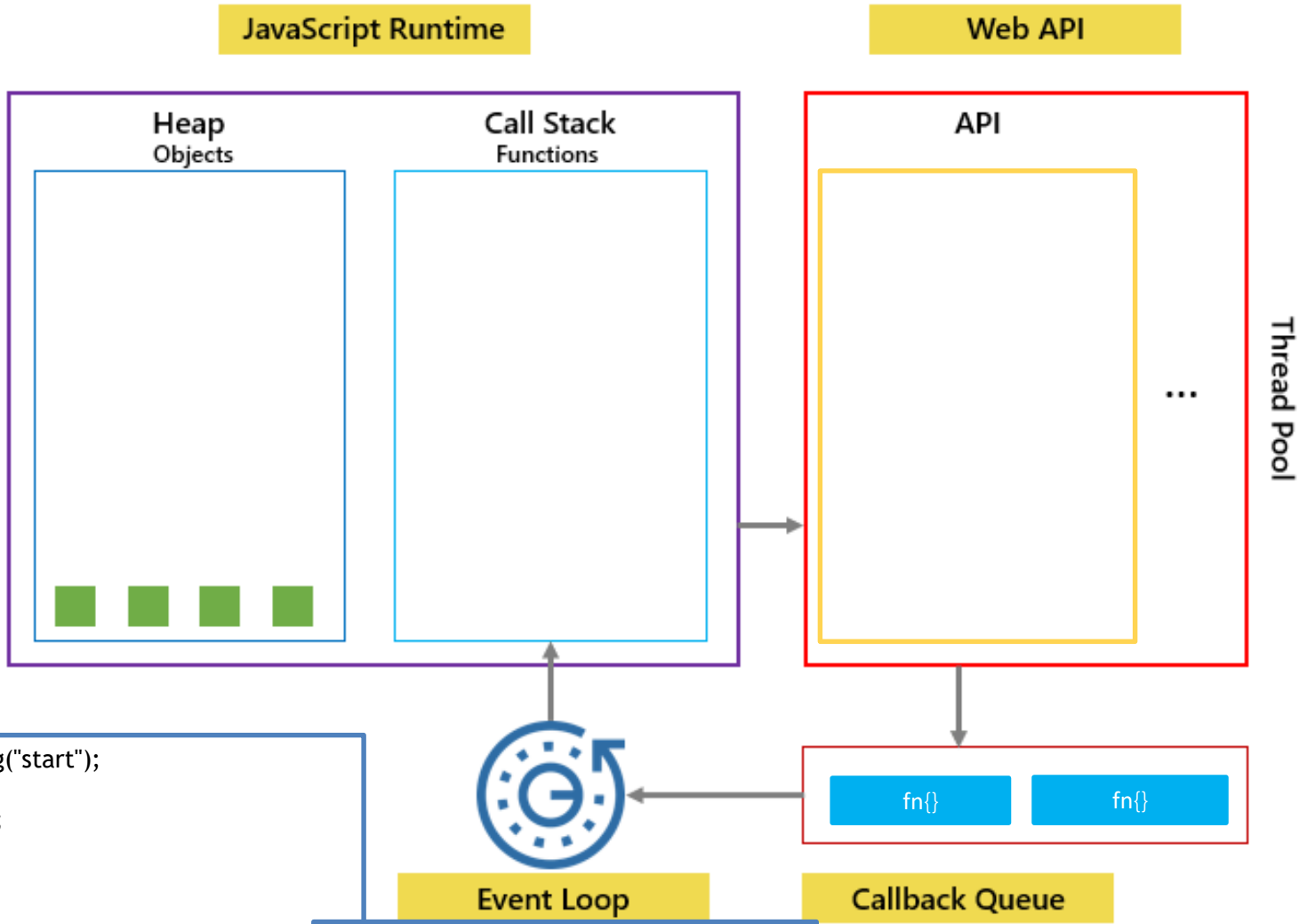

```
console.log("start");

wait5sec();

fun();

console.log("end");

setTimeout(function () {
    console.log("timeout after 1 sec");
}, 1000)
```

```
function fun() {
    setTimeout(function () {
        console.log("timeout immediately");
    }, 0);
}
```



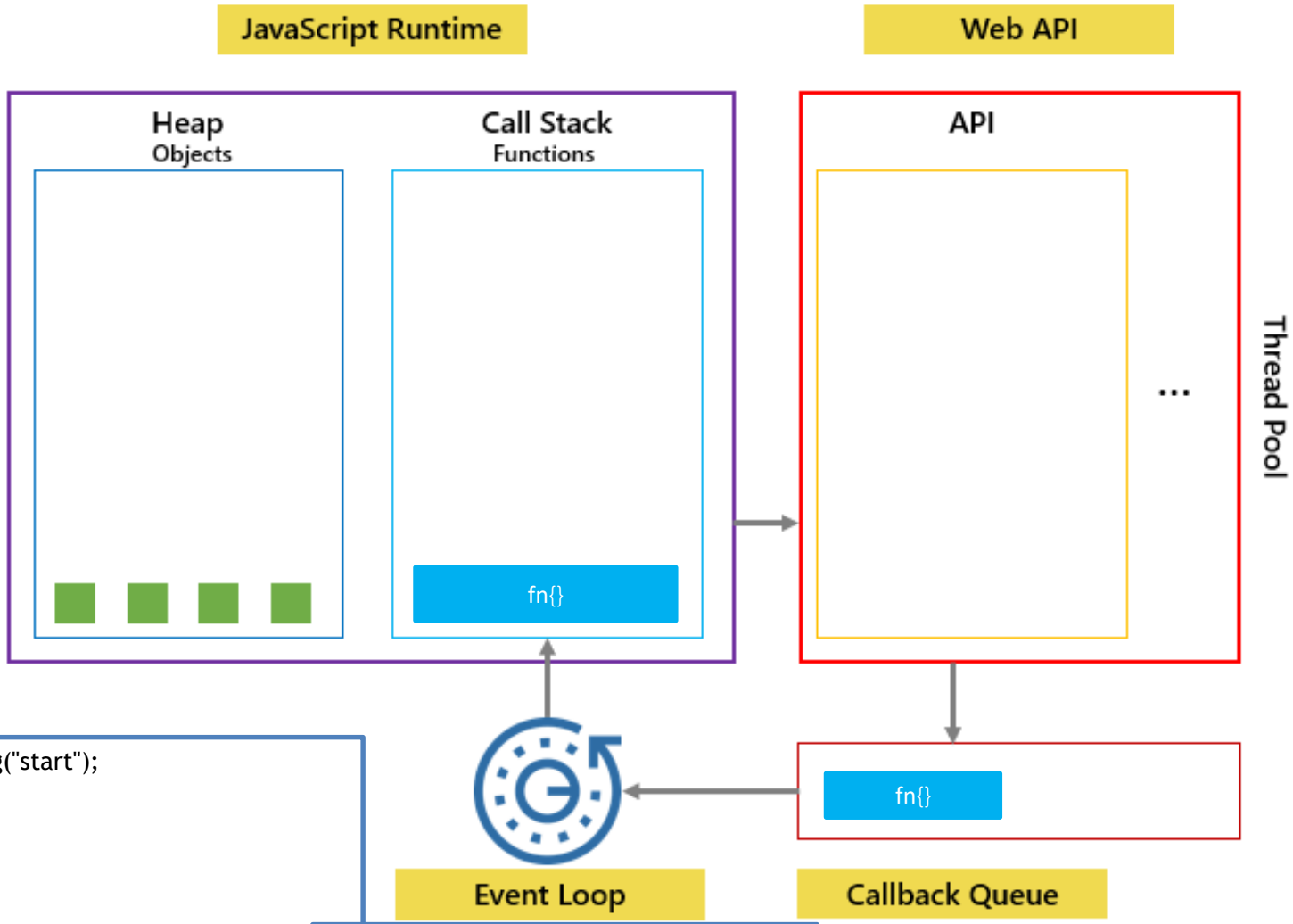

```
console.log("start");

wait5sec();

fun();

console.log("end");

setTimeout(function () {
    console.log("timeout after 1 sec");
}, 1000)
```

```
function fun() {
    setTimeout(function () {
        console.log("timeout immediately");
    }, 0);
}
```




```
console.log("start");

wait5sec();

fun();

console.log("end");

setTimeout(function () {
    console.log("timeout after 1 sec");
}, 1000)
```

```
function fun() {
    setTimeout(function () {
        console.log("timeout immediately");
    }, 0);
}
```




```
console.log("start");

wait5sec();

fun();

console.log("end");

setTimeout(function () {
    console.log("timeout after 1 sec");
}, 1000)
```

```
function fun() {
    setTimeout(function () {
        console.log("timeout immediately");
    }, 0);
}
```

JavaScript is the single-threaded programming language.

The JavaScript engine has only one call stack so that it only can do one thing at a time.

JavaScript execution is synchronous.
When executing a script,
the JavaScript engine executes code
from top to bottom,
line by line.

JavaScript also supports asynchronous operations through mechanisms like callbacks, events etc.

event loop to handle asynchronous operations

asynchronous nature is essential for handling non-blocking operations

Navigator

- The navigator object represents the browser application.
- This object allows scripts to get information about the browser like its type, version, language etc..
- Object Model reference:

  [window.]navigator
- All of its properties are read-only.

Navigator Properties & Methods

- Methods:
  - **javaEnabled()**
- Properties

https://developer.mozilla.org/en-US/docs/Web/API/Navigator

Name	Description	Syntax
appName (deprecated)	get the name of the browser	navigator.appName
appVersion (deprecated)	get the version of the browser	navigator.appVersion
language	get the default language of the browser	navigator.language
cookieEnabled	returns whether the browser allows cookies or not	navigator. cookieEnabled
platform (deprecated)	return the name of the OS	navigator.platform
vendor (deprecated)	Returns the vendor name of the current browser	navigator.vendor

HTML5 New API →

geolocation

returns a Geolocation object allowing accessing the location of the device

Example!

Location

- The Location object is part of a Window object.
- The location Object refers to the current URL.
- Location contains information about the current URL of the browser. The most common usage of Location is simply to use it to automatically navigate (redirect) the user to another web page.
- It has a set of properties to hold the different components of the URL
- Object Model Reference:

  [window.]location

```
<script type="text/javascript">
    window.location=http://www.google.com
< /script>
```

Location Properties

Name	Description	Syntax
href	is the default property of the location object, returns the entire URL	location.href
protocol	represents the protocol of the URL.	location.protocol
hostname	specifies the host name	location.hostname
port	specifies the communication port.	location.port
host	is a combination of the host name and port	location.host
pathname	is the directory to find the document on the host, and the name of the file	location.pathname
search	specifies the queryString	location.search

Location Methods

- *replace* method loads the specified URL over the current history entry.

  location.replace(URL)

- *reload* method Reloads the current document over the current history entry.

  location.reload()

- *assign* method is almost the same as *replace* method.
  The difference is that it creates an entry in the browser's history list, while replace() doesn't.

  location.assign(URL)

- *toString* method returns a string representation containing the whole URL

  location.*toString* ()

Example!

History

- The history Object lets you send the user to somewhere in the history list from within a JavaScript program.
- Object Model reference:

  [window.]history

- Properties:

length

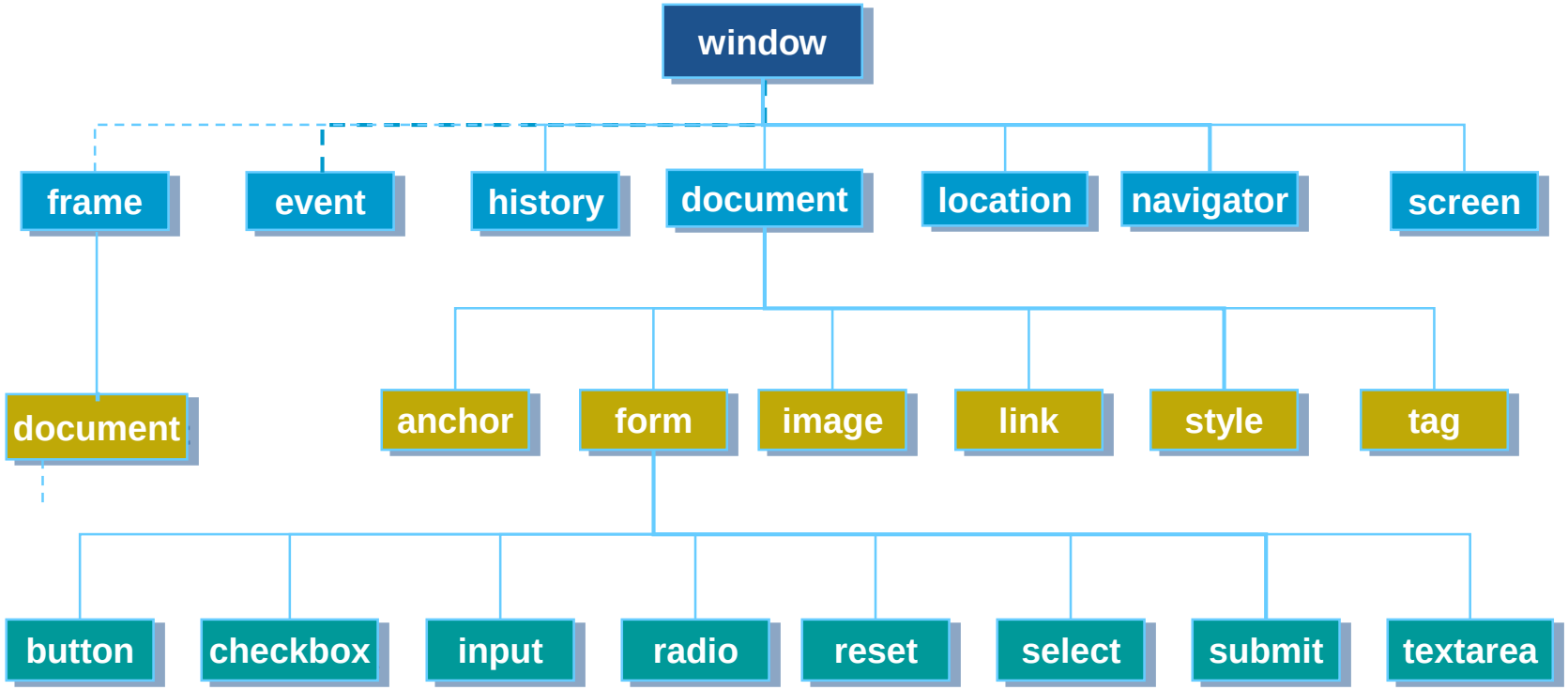
- Methods:

back()	**forward()**	**go()**

Example!

Model Hierarchy

BOM is a larger representation of everything provided by the browser including any other functionality the browser may expose to JavaScript.



Document Object

- The *document* object represents the entire HTML document and can be used to access all elements in a page.
- A *page* is what appears within the browser window.
- So, every window is associated with a document object.
- Document Object has its own set of Properties, Collections, Methods & Event handlers.

DOM & document Object

- The **document** object in the **BOM** is the top level of the **DOM** hierarchy.
- DOM is a representation of the whole document as nodes and attributes.
- You can access each of these nodes and attributes and change or remove them.
- You can also create new ones or add attributes to existing ones.

DOM is a subset of BOM.

In other word: **the document is yours!**

DOM

- DOM Stands for Document Object Model.
- W3C standard.

https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

- Its an API that interact with documents like HTML, XML.. etc.
- Defines a standard way to access and manipulate HTML documents.
- Platform independent.
- Language independent

Document Methods

BOM

Method	Description
write()	Writes one or more HTML expressions to a document in the specified window.
writeln()	Writes one or more HTML expressions to a document in the specified window and follows them with a new line character.

Example!

Document Properties

Property	Description
bgColor	A string that specifies the background color.
fgColor	A string that specifies the text color.
linkColor	The color of text for a link that the user has not yet visited.
vlinkColor	The color of text for a link that the user has already visited.
alinkColor	The color of text for a link that the user clicks.
title	A string that specifies the contents of the TITLE tag.
cookie	A string containing the name/value pair of cookies in the document.

The document object in the BOM is the top level of the DOM hierarchy.

DOM

methods and properties, allows accessing any element on the page, modify, delete or remove elements, or add new ones using **document** object

Document Methods

W3C

for accessing document elements

Method	Description
getElementById(“id")	For accessing any element on the page via its ID attribute
getElementsByName(“name”)	Returns a collection of objects with the specified name
getElementsByTagName(“tagName”)	Returns a collection of objects with the specified tagname

Example!

New HTML5 Selectors

W3C

- In HTML5 we can select elements by ClassName

```
var elements = document.getElementsByClassName('entry');
```

- Moreover there's now possibility to fetch elements that match provided CSS syntax

```
var elements =  document.querySelectorAll(".someClasses");
```

```
var elements = document.querySelectorAll("div,p");
```

```
var elements = document.querySelector("#someID");
```

```
var first_td =   document.querySelector("span");
```

DOM Accessing method returns a JavaScript object that represents the HTML markup element. This Object's properties represent the markup's attribute

We can access any markup content via innerHTML, innerText, or textContent properties

Document Collection

- links, anchors, images, forms are collection/array containing all occurrences of those objects within the document.
- Since they are treated as arrays, they have the *length* property which specifies the number of entries in the collection/array.
- To access a specific entry in any of these collections, we can use either their index or name.

Document Collection

Collection	Description
forms[ ]	An array containing an entry for each form in the document
images[ ]	An array containing an entry for each image in the document
anchors[ ]	An array containing an entry for each anchor in the document.
links[ ]	An array containing an entry for each link in the document.

Document Collection

W3C

- An item from an object collection can be referenced in one of the following ways:

  1. collection[i]
  2. collection.item(i)
  3. collection.namedItem(id)
  4. collection[“name”]
  5. collection[“id”]
  6. collection.name

- Example: document.forms[0]
  document.forms[“myForm”]
  document.forms[“frmid”]
  document.myForm

Document Event Handler

onclick
ondblclick
onkeydown
onkeypress
onkeyup
onmousedown
onmouseup

Image

- The Image object is an image on an HTML form, created by using the HTML 'IMG' tag.
- Any images created in a document are then stored in an array in the document.images property.
- Image Object has its own set of Properties, Methods & Event handlers.

Image

- **Object Model Reference:**

  [window.]document.imageName
  [window.]document.imageID
  [window.]document.images[i]
  [window.]document.images[imageName]

```
<html>
<body>
 …..
<img name="img1" src="img1.jpg">
<img name="img2" src="img2.jpg">
…
</body>
</html>
```

```
document.img1.src="img1.jpg"
document.images[0].src="img1.jpg"
```

```
document.img2.src="img2.jpg"
document.images[1].src="img2.jpg"
```

Image Properties & Event handlers

Properties

name	id	src	height	width

Event handlers

onabort	onload	onerror	onclick	ondblclick	onmouseover

Example!

Assignments